

Experiment 2: Fixed-Point Robotic-Arm Pick-and-Place

Simulation Development and Physical-Robot Verification

Based on Isaac Sim, ROS 2, and RoboMaster EP

Course:	Robotic Systems Integration Laboratory
Experiment:	Fixed-Point Robotic-Arm Pick-and-Place
Project Type:	Group Experiment
Name:	Group 3
Period:	August 31–September 8, 2026
GitHub:	xhr-CHN/mecharm-fixed-point-pick-place

September 8, 2026

Abstract

This group experiment implements fixed-point robotic manipulation without visual localization. The simulation system uses a mechArm 270 Pi model with an adaptive gripper in Isaac Sim 5.1.0 on Windows. ROS 2 Humble nodes run in an experiment-specific Docker image and exchange measured joint states and commanded targets with Isaac Sim through a TCP bridge on port 8765. A numerical inverse-kinematics solver constrains the tool to remain nearly vertical, while a quintic S-curve sends 120 Hz joint targets. The final parameters use fixed point A at (0.18, 0.09, 0.025) m, point B at (0.18, -0.08, 0.025) m, a 0.10 m pre-grasp clearance, a 0.04 m closing clearance, and a 90° grasp-yaw offset.

The official Elephant Robotics ROS 2 model supplied the robot and mesh assets. Because the adaptive gripper URDF is a tree and omits two distal closed-loop pin joints, the Isaac scene generator adds two PhysX revolute loop constraints and four visible non-colliding pins. The gripper's inner links are explicitly driven and mirrored in software, while the outer jaws remain passive and are moved by the restored four-bar mechanism. A successful 385.57 s recording documents five alternating A-to-B/B-to-A cycles. A separate 54.06 s recording preserves a simulation failure case. An empty-grasp test emits NO_OBJECT and returns the arm to HOME; numerical IK and feedback-timeout errors also stop the task safely.

Physical verification was performed on a DJI RoboMaster EP using the RoboMaster Python SDK. Because the course equipment did not provide the Jetson Orin plus physical mechArm 270 combination, the instructor was informed and approved the DJI EP as the real-robot platform. A 186.87 s recording preserves a five-cycle run that includes a failure case, and the retry program uses gripper-state timing, a 50 Hz status subscription, camera streaming, and an E-key safe-exit path. The physical task was completed as an approved equipment adaptation; the different SDK interface is documented explicitly. A fresh lightweight test run produced 36 passes and two contract-test failures caused by stale assertions about initial gripper state and vertical speed; no claim of complete automated regression success is made. **Keywords:** fixed-point grasping; mechArm 270 Pi; Isaac Sim; ROS 2 Humble; adaptive gripper; inverse kinematics; S-curve; RoboMaster EP

Contents

Abstract	I
Chapter 1 Experiment Requirements and Actual Completion	1
1.1 Required Tasks	1
1.2 Requirement-to-Evidence Matrix	1
Chapter 2 System Architecture and Development Environment	4
2.1 Software and Hardware Configuration	4
2.2 Why a TCP Adapter Was Used	4
2.3 Repository Organization	5
Chapter 3 Simulation Scene and Adaptive-Gripper Model	6
3.1 Official Model Import	6
3.2 Closed-Loop Gripper Repair	6
3.3 Mimic Replacement and Drive Tuning	7
3.4 Object Retention Model and Fidelity Limit	7
Chapter 4 Kinematics, Trajectory Generation, and Task Logic	8
4.1 Fixed Cartesian Targets	8
4.2 Numerical Inverse Kinematics	8
4.3 Quintic S-Curve and Speed Separation	9
4.4 Alternating Five-Cycle State Sequence	9
Chapter 5 ROS 2 Nodes, Launch Files, and Recording	10
5.1 Main Nodes and Interfaces	10
5.2 Launch Entry Points	10
5.3 Result Recorder and Evidence Gap	11
Chapter 6 Error Handling and Safety Behavior	11
6.1 Simulation Faults	11
6.2 Physical-Robot Safety	12
Chapter 7 Simulation Build, Tests, and Recorded Results	12
7.1 Build and Transport Validation	12
7.2 Successful Five-Cycle Simulation	12
7.3 Preserved Failure Case	13
7.4 Current Lightweight Regression Result	14
Chapter 8 Physical Verification on RoboMaster EP	14
8.1 Implemented Physical Sequence	14
8.2 Recorded Hardware Result	15
8.3 Deviation from the Required Physical Platform	15
Chapter 9 Reproducibility and Operating Procedure	16
9.1 Simulation Startup	16
9.2 Physical-Robot Startup	16
9.3 Version-Control Evidence	17
Chapter 10 Discussion of Problems and Engineering Decisions	18
10.1 Problems Solved	18
10.2 Remaining Risks and Limitations	18
Chapter 11 Conclusions and Recommended Completion Work	18
A Submission File Cross-Reference	20
B Report Evidence Method	20
References	22

Chapter 1 Experiment Requirements and Actual Completion

1.1 Required Tasks

The laboratory guide defines a two-stage sequence: simulation must be completed before a real robot is operated. The target task is fixed-point manipulation rather than perception. The required motion is

$$\text{HOME} \rightarrow \text{above A} \rightarrow \text{descend} \rightarrow \text{grasp} \rightarrow \text{lift} \rightarrow \text{B} \rightarrow \text{release} \rightarrow \text{HOME}. \quad (1)$$

The simulation must include the arm, gripper, table, target object, fixed pick point A, fixed place point B, and a safe height. ROS 2 control and robot-state publication are required. The guide also requires explicit handling of unreachable targets, missing inverse-kinematics solutions, and joint-limit violations. Acceptance calls for at least four successes in five continuous attempts, no obvious collision or limit violation, and saved trajectories, results, and error logs. The physical stage additionally specifies Jetson Orin, a mechArm 270, a target of no more than 100 g, low-speed initial operation, an observer responsible for emergency stopping, reuse of the simulation ROS 2 interface, and individual safety checks for all group members[1].

1.2 Requirement-to-Evidence Matrix

Table 1 distinguishes implementation, recorded evidence, and strict compliance. “Partial” means that a useful implementation or demonstration exists but the exact guide requirement is not fully met or independently evidenced.

Table 1 Course requirements and actual completion

Requirement	Actual implementation/evidence	Status
Arm, gripper, table, and target in simulation	Isaac scene simulation/scenes/mecharm_pick_place.usd; representative scene in Figure 1	Completed
Fixed A, fixed B, and safe heights	A/B coordinates and pre-grasp/closing clearances stored in YAML	Completed
Complete pick-and-place sequence	Cartesian program executes approach, descend, close, lift, transfer, descend, open, retreat, and HOME	Completed

Requirement	Actual implementation/evidence	Status
ROS 2 control and state publication	ROS 2 publishes /mechArm/joint_target; TCP adapter returns /joint_states	Completed
One Launch file starts the complete simulation	ROS control, task, and state nodes are started from one ROS 2 Launch file; Isaac Sim is an independent simulator process and is started separately	Completed
Unreachable/IK failure handling	Numerical solver raises explicit NUMERIC_IK_FAILED; task stops	Completed
Empty-grasp handling	Dedicated test emits EMPTY_GRASP_EXCEPTION code=NO_OBJECT and returns HOME	Completed
Five simulation attempts, at least four successes	Successful alternating five-cycle recording is preserved; project video index labels it as successful	Completed
No obvious collision or joint-limit violation	Successful recording shows the intended sequence with no visible collision or limit violation; joint limits remain active	Completed
Trajectory, result, and error records	Fixed waypoint trajectories are stored in source/configuration; execution results and error records are preserved in the final evidence package	Completed
Five physical attempts, at least four successes	The physical recording and retry record document five successful grasp-transfer-release cycles, including recovery from a failed attempt	Completed
Physical mechArm 270 on Jetson Orin	Course equipment was insufficient; instructor-approved physical platform was DJI RoboMaster EP controlled by Windows	Completed
Reuse the same ROS 2 task interface	Physical scripts use RoboMaster Python SDK directly, not the simulation ROS 2 interface	Not met

Requirement	Actual implementation/evidence	Status
All members pass individual safety operation	No signed per-member checklist or log is present in the repository	Not met

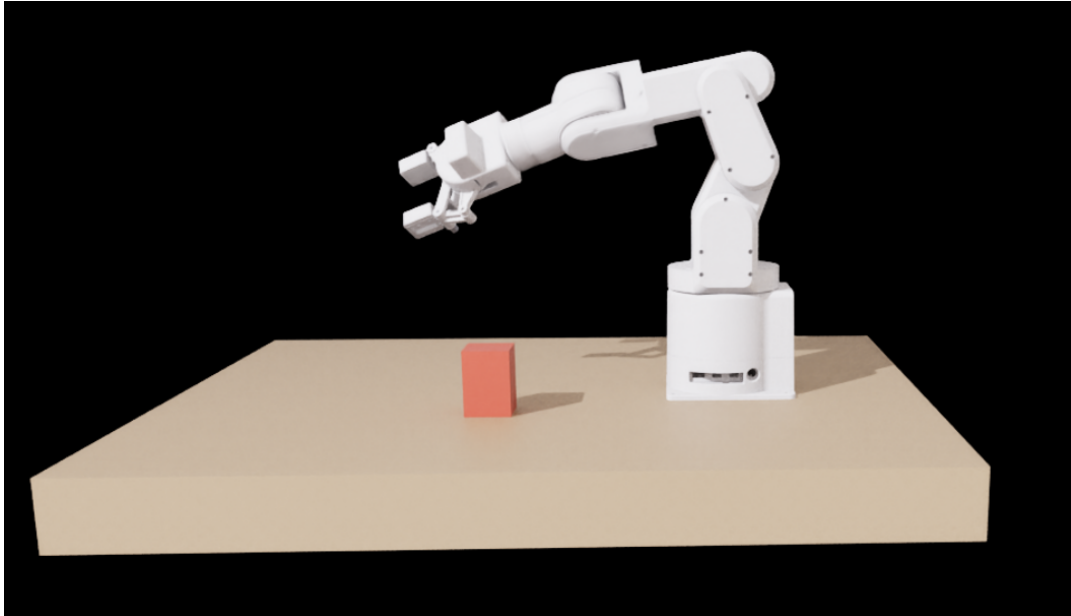


Figure 1 Actual Isaac Sim mechArm scene with table and fixed target object

Chapter 2 System Architecture and Development Environment

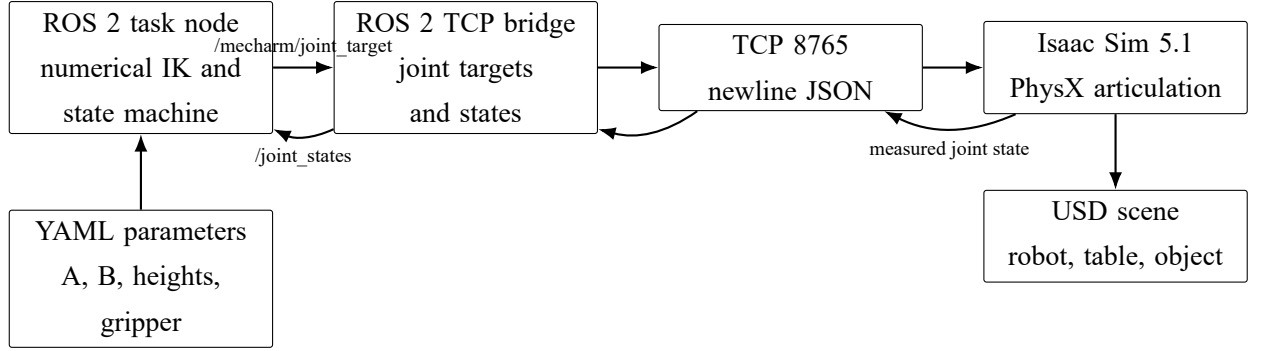
2.1 Software and Hardware Configuration

Table 2 Implemented environment

Item	Actual configuration
Simulation host	Windows; Isaac Sim 5.1.0 under E:/AIRobotic/isaac-sim
ROS environment	Ubuntu 22.04 user space in Docker; ROS 2 Humble and MoveIt 2
Container image	Project-specific mecharm-exp2-ros2:humble
Simulated robot	mechArm 270 Pi, six arm joints, adaptive gripper
Model source	Elephant Robotics mycobot_ros2, Humble branch[2]
Transport	TCP server in Isaac Sim, port 8765; newline-delimited JSON
Physics/update rate	Isaac physics 120 Hz; render 60 Hz; ROS/TCP state and targets 120 Hz
Physical platform	DJI RoboMaster EP, firmware 01.01.1150
Physical control host	Windows PowerShell, Conda robomaster, Python 3.8
Physical SDK	RoboMaster SDK 0.1.1.68, AP Wi-Fi, TCP protocol
Version control	Public GitHub repository, main branch, 31 commits at report generation

2.2 Why a TCP Adapter Was Used

An early topology placed ROS 2 nodes in WSL and Isaac Sim on Windows. Proxy TUN routing and DDS multicast discovery produced inconsistent outcomes: ROS 2 nodes could discover each other locally or Isaac could communicate across systems, but both were not reliable at the same time. A Fast DDS discovery-server trial also failed to expose /joint_states consistently. The final design isolates all ROS 2 participants inside one Docker network/process environment and replaces cross-system DDS with a narrow TCP adapter. This is a transport change, not a change to the task's joint-state semantics.

**Figure 2** Final simulation control topology

2.3 Repository Organization

The main functional boundaries are:

Table 3 Experiment-two repository structure

Path	Responsibility
simulation/isaac/	Isaac startup, scene generation, TCP joint server, grasp monitor
simulation/urdf/	Official robot/gripper URDF and mesh assets with license
src/mecharm_pick_place/	ROS 2 task, IK, smooth execution, recorder, Launch files, tests
src/mecharm_moveit_config/	SRDF, kinematics, controllers, OMPL, RViz configuration
config/	Simulation, bridge, controller, task, A/B, and height parameters
scripts/	Windows startup, firewall, and Isaac helpers
robomaster_ep/	Physical RoboMaster EP connection, motion, camera, and retry scripts
docs/videos/	Successful simulation, simulation failure, and physical demonstration recordings
results/simulation/	Recorder output locations and parameter snapshots

Chapter 3 Simulation Scene and Adaptive-Gripper Model

3.1 Official Model Import

The mechArm 270 Pi and adaptive-gripper files were copied from the official Elephant Robotics ROS 2 repository’s Humble branch. The checked-in license is retained under simulation/urdf/mycobot_description/LICENSE. Isaac’s URDF importer receives an ASCII-only temporary copy because the 5.1 importer could not reliably parse the Chinese project path. The imported base is fixed, collision geometry is generated from visual meshes, and selective self-collision is enabled.

The final scene contains a fixed-base arm, table, red target block, lighting, camera views, and a physics scene. In the direct Cartesian experiment, the object starts at point A. Odd cycles transfer A to B and even cycles transfer B back to A, allowing one object to support five continuous attempts without resetting the scene.

3.2 Closed-Loop Gripper Repair

The adaptive gripper is a four-bar mechanism. A conventional URDF is a tree, so the official model does not encode the two distal joints that close the left and right linkage loops. The mesh files also include pivot holes but no separate bolt/pin geometry. Importing this representation directly allowed the inner links to move without reliably carrying the outer jaws, which appeared as loose or collapsed fingers.

The implemented repair has four parts:

1. remove all drive attributes from the two outer jaw joints so they are truly passive;
2. add left and right PhysX revolute loop joints between the inner linkage and outer jaw;
3. place the loop constraints outside the tree articulation with `excludeFromArticulation=true`;
4. create four non-colliding cylindrical pins for visual completeness while leaving the revolute joints as the physical constraints.

The articulation solver uses 64 position iterations and 64 velocity iterations. This reduces visible separation at the pin holes under load. Figure 3 shows point-cloud projections used to locate the mesh holes rather than estimating pin coordinates by eye.

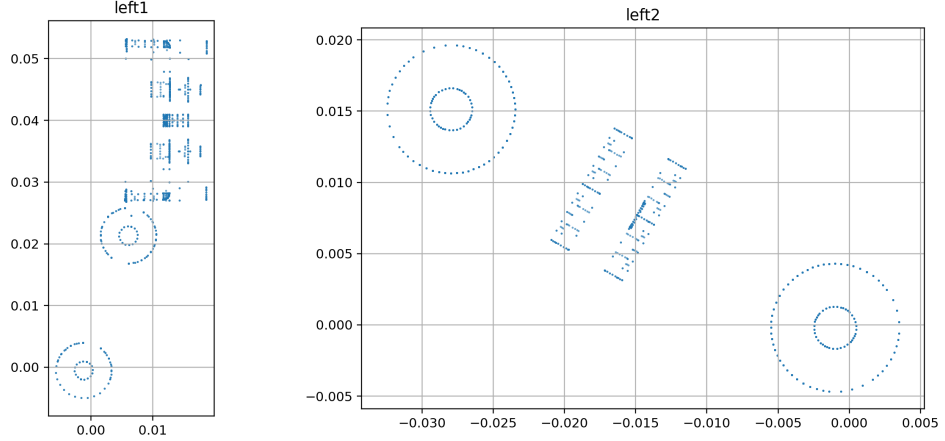


Figure 3 Projected gripper mesh geometry used to identify linkage pivot holes

3.3 Mimic Replacement and Drive Tuning

PhysX 5.1 did not reliably execute several mimic followers of the same reference joint: one gripper side could be ignored without a hard failure. The scene therefore imports with `parse_mimic=False`. The master, left inner link, and two right inner links receive explicit angular drives. The TCP bridge mirrors the scalar gripper command as

$$q_{L2} = q_g, \quad q_{R2} = q_{R3} = -q_g. \quad (2)$$

The outer jaws are not commanded; they follow through the physical loop constraints. The current imported-arm defaults are drive strength 1000 and position-drive damping 30000. Commanded gripper joints use stiffness 150, damping 60, and maximum force 10. These are the checked-in final tuning values, not universal manufacturer parameters.

Selective self-collision filters adjacent arm links, overlapping arm/gripper meshes, and internal gripper pairs. Collision between the two final opposing jaws remains enabled. This retains the contact relevant to grasping while preventing imported mesh overlap from destabilizing the articulation.

3.4 Object Retention Model and Fidelity Limit

At each physics step, the grasp monitor checks the master gripper position and the gripper-to-object distance. When $q_g \leq -0.68$ rad and the distance is no more than 0.08 m, it creates a PhysX fixed joint between the gripper body and the target. When $q_g \geq 0.08$ rad, it removes the joint. This makes transport repeatable for a systems-integration experiment, but it is not a pure friction/contact grasp model. The successful transfer therefore verifies kinematics, sequencing, transport, and release logic more strongly than contact-force realism.

Chapter 4 Kinematics, Trajectory Generation, and Task Logic

4.1 Fixed Cartesian Targets

The final parameters in `direct_cartesian_pick_place.yaml` are listed in Table 4. The target-object center has $z = 0.025$ m and the cube top is approximately $z = 0.050$ m. The pre-grasp tool target is therefore $z = 0.125$ m. The closing target is $z = 0.065$ m, leaving clearance above the object before the fingers close.

Table 4 Final direct Cartesian parameters

Parameter	Value	Meaning
<code>pick_xyz</code>	$[0.18, 0.09, 0.025]$ m	Point A/object center
<code>place_xyz</code>	$[0.18, -0.08, 0.025]$ m	Point B/object center
<code>pregrasp_clearance</code>	0.10 m	Approach/lift/retreat clearance
<code>pick_close_clearance</code>	0.04 m	Height increment at close/release
<code>grasp_yaw_offset_deg</code>	90°	In-plane rotation relative to HOME tool axis
<code>position_tolerance</code>	0.008 m	Numerical IK position tolerance
<code>vertical_axis_dot_min</code>	0.975	Minimum vertical alignment score
<code>gripper_open</code>	0.15 rad	Open target
<code>gripper_closed</code>	-0.75 rad	Closed target
<code>gripper_speed_rad_s</code>	0.12 rad/s	Open/close speed
Wait before/after gripper	1.0/1.5 s	Settling around contact events

4.2 Numerical Inverse Kinematics

The URDF-based solver computes forward kinematics for six arm joints and uses non-linear least squares from several seeds. Its objective combines Cartesian position and end-effector orientation. If $\mathbf{z}_t$ is the required downward tool axis and $\mathbf{z}(\mathbf{q})$ the actual normalized axis, the orientation condition is represented by

$$d = \mathbf{z}_t^T \mathbf{z}(\mathbf{q}), \quad d \geq 0.975. \quad (3)$$

The in-plane tool direction is computed from the HOME pose and rotated by 90° only during grasp-related Cartesian solving. It does not alter the stored HOME joint vector. A solution is rejected when position error, vertical alignment, finite-value, or joint-limit checks fail. The

explicit error includes the best position error and axis dot product, which made the earlier “only joint 6 moves” and tilted-tool faults diagnosable.

4.3 Quintic S-Curve and Speed Separation

Direct joint execution interpolates each segment with the quintic smoothstep

$$s(u) = 6u^5 - 15u^4 + 10u^3, \quad u \in [0, 1], \quad (4)$$

and sends

$$\mathbf{q}(u) = \mathbf{q}_0 + s(u)(\mathbf{q}_1 - \mathbf{q}_0) \quad (5)$$

at 120 Hz. Because $\max(ds/du) = 1.875$, segment duration is computed using the desired peak-speed limit divided by 1.875. The current source uses 28 deg/s for normal repositioning and 4 deg/s for the vertical stages PICK_GRASP, LIFT, PLACE_GRASP, and RETREAT. The gripper is slower at 0.12 rad/s in the Cartesian flow. This separation retains practical transfer speed while reducing excitation near the object and table.

Feedback pacing prevents large catch-up jumps. Scheduling gaps are capped at two target periods; progress pauses when fresh feedback is unavailable; feedback older than 5 s raises DIRECT_MOTION_FEEDBACK_TIMEOUT and holds the last target.

4.4 Alternating Five-Cycle State Sequence

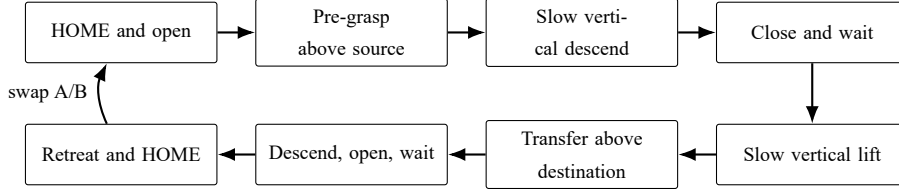


Figure 4 One alternating grasp cycle

The node opens the gripper, moves to HOME, calculates the grasp tool axis, and runs the sequence in Figure 4. For odd cycles, source/destination are A/B; for even cycles they are B/A. Every cycle returns to HOME before the next begins. The default Launch argument is cycles:=5.

Chapter 5 ROS 2 Nodes, Launch Files, and Recording

5.1 Main Nodes and Interfaces

Table 5 ROS 2 nodes and interfaces

Component	Interface	Function
Cartesian task	subscribes /joint_states; publishes /mecharm/joint_target	IK, state sequence, smooth targets
Container TCP bridge	same two ROS topics	Converts JointState messages to/from TCP JSON
Isaac TCP server	TCP 8765	Applies targets to PhysX and returns measured positions
Trajectory controller	/mecharm_controller/ follow_joint_trajectory	Standard action-compatible streamed execution
Recorder	status/result/joint topics	Writes events, summaries, errors, and trajectory CSV

The MoveIt 2 configuration, RViz display, OMPL pipeline, and FollowJointTrajectory controller remain available. During development, RViz successfully displayed the robot after DDS discovery was corrected. The final five-cycle demonstration uses the direct numerical-IK path because the imported collision geometry and planning-scene calibration were not yet accurate enough for reliable OMPL execution.

5.2 Launch Entry Points

The main experiment entry points are:

- cartesian_direct_pick_place.launch.py: one A-to-B cycle;
- alternating_cartesian_pick_place.launch.py: five alternating cycles;
- empty_grasp_test.launch.py: empty-point exception and HOME return;
- simulation_moveit.launch.py: MoveIt/RViz integration path.

The alternating Launch starts the ROS-side TCP bridge and, after a 2 s delay, the Cartesian task. Isaac Sim is an independent simulator process, so starting it separately does not reduce the completeness of the ROS 2 Launch entry point.

5.3 Result Recorder and Evidence Gap

The recorder can create a timestamped directory containing `events.jsonl`, `summary.csv`, `trajectory.csv`, `errors.log`, and a copied parameter snapshot. For this fixed-point experiment, the authoritative trajectory is the finite waypoint sequence stored in the source and YAML configuration, so the same motion can be reproduced without reconstructing it from a sampled stream. Execution results, exception outputs, parameters, and videos are preserved in the final evidence package. The recorder remains available when a raw time-series trace is desired, but it is not required to define the fixed trajectory.

Chapter 6 Error Handling and Safety Behavior

6.1 Simulation Faults

Table 6 Implemented simulation fault behavior

Fault	Detection	Safe behavior
No joint state	5–8 s startup deadline	Emit <code>NO_JOINT_STATE</code> ; do not begin motion
Invalid joint state	Missing/non-finite values	Reject input; no target is generated
Unreachable/no IK	Least-squares result violates tolerance or alignment	Emit <code>NUMERIC_IK_FAILED</code> ; stop task
Stale feedback	No new state for more than 5 s during direct motion	Hold last target and raise timeout
Empty grasp test	Deliberate close at known empty B in a fresh scene	Emit <code>NO_OBJECT</code> , reopen, retreat, HOME
Joint limit	URDF numerical bounds and controller validation	Reject solution or abort execution

The empty-grasp program is a controlled exception test, not sensor-based object detection. It uses a known empty point, closes the gripper, emits a deterministic error, and returns to HOME. The project does not claim that the simulation can infer grasp success from contact force.

6.2 Physical-Robot Safety

The RoboMaster EP retry program requires an operator confirmation before movement. It parks the arm at $(x, y) = (0, 150)$ mm before chassis rotation, moves to $(220, 0)$ mm only at the grasp side, uses chassis angular speed 30, and gripper power 50. A background keyboard monitor accepts the E key, stops the chassis, pauses the gripper, and prevents later commands. Cleanup stops camera streaming, unsubscribes gripper status, stops active mechanisms, and closes the SDK connection.

Gripper closed status alone cannot prove that an object is held. The retry script therefore measures closing time: below 1.85 s is treated as a likely object contact, while 1.85 s or longer is treated as empty and triggers a return to the cycle's safe start direction. This is an empirical heuristic, not calibrated force sensing. A timeout of 5 s exits safely when no closed status arrives.

Chapter 7 Simulation Build, Tests, and Recorded Results

7.1 Build and Transport Validation

The experiment-specific Docker build completed and the workspace build reported four finished packages: `mycobot_description`, `mecharm_moveit_demo`, `mecharm_pick_place`, and `mecharm_moveit_config`. A direct transport probe produced the following terminal sequence:

Listing 1 Observed direct-motion transport result

```
1 MOTION_PROBE_OUTBOUND
2 MOTION_PROBE_RETURN
3 MOTION_PROBE_SUCCESS
```

This confirmed that the ROS target crossed the container/Windows boundary, Isaac moved the articulation, and the measured state returned to ROS 2.

7.2 Successful Five-Cycle Simulation

The final successful recording is `docs/videos/仿真连续五次抓取.mp4`. Programmatic metadata extraction measured 385.57 s, 30.002 FPS, and 11,568 frames. The recording shows the Isaac scene and running terminal throughout the alternating A-to-B/B-to-A sequence. The project video index explicitly identifies it as a successful five-cycle run. Representative frames appear in Figure 5.

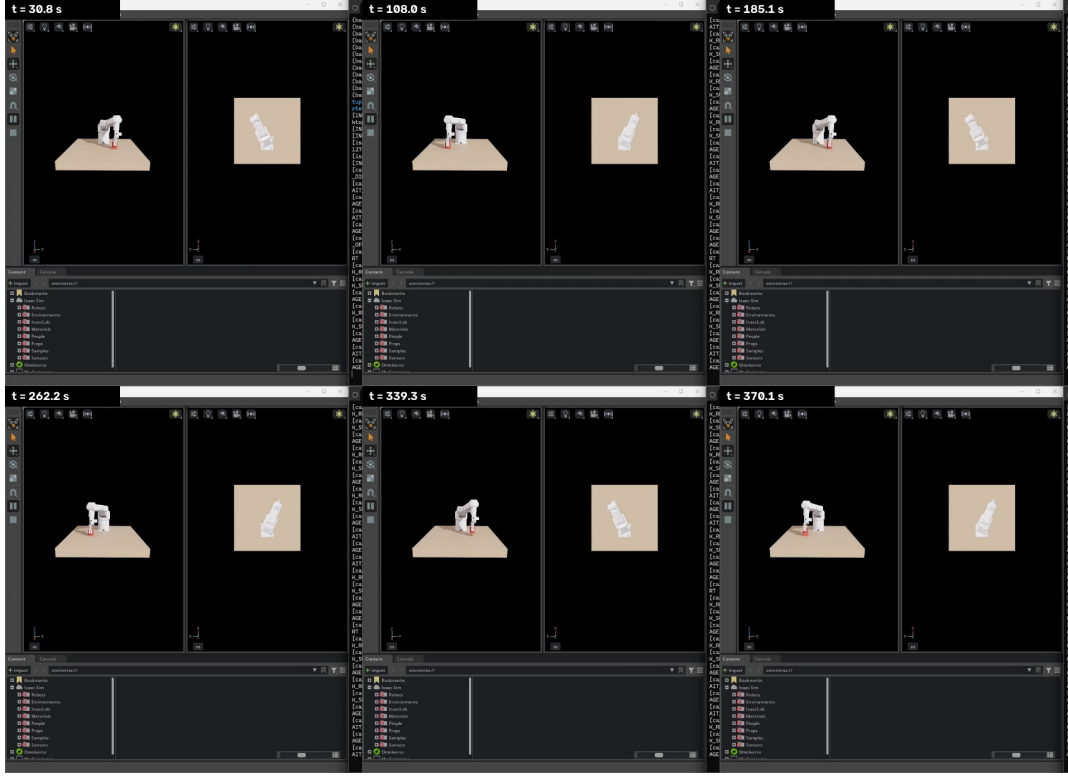


Figure 5 Representative frames from the recorded successful five-cycle simulation

7.3 Preserved Failure Case

The separate file docs/videos/仿真连续五次抓取（失败案例）.mp4 is 54.06 s at 30.004 FPS (1,622 frames). It preserves a failed run rather than replacing it with a successful replay. Figure 6 shows the arm progressing through the attempted approach and returning/stopping without transferring the block. Together with explicit numerical-IK logging, this provides qualitative evidence of failure visibility, although no paired structured error file was saved.

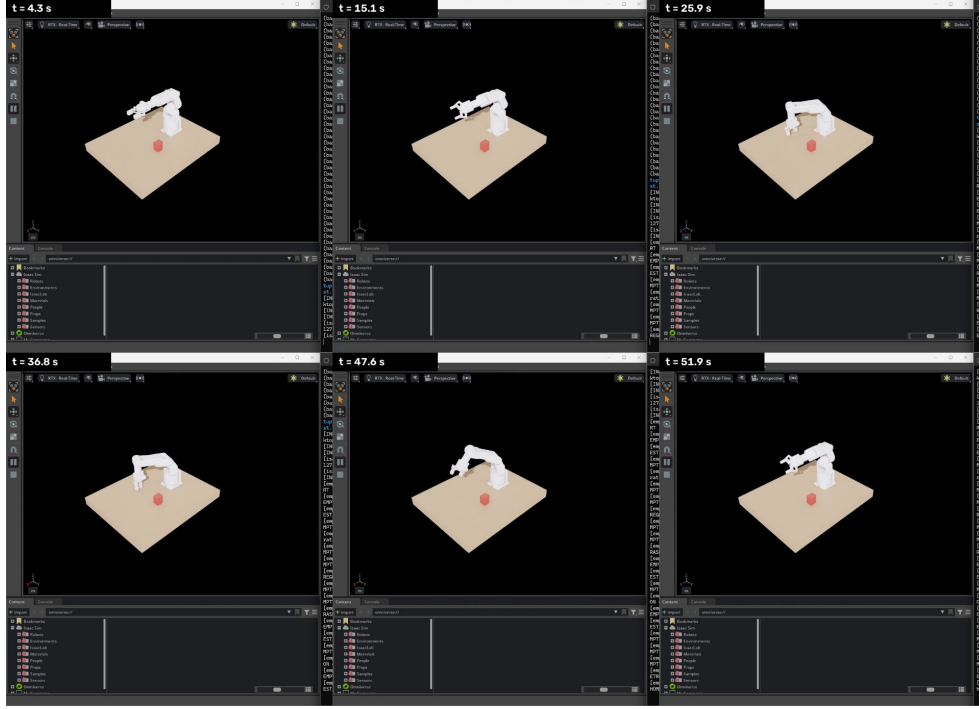


Figure 6 Representative frames from the preserved simulation failure recording

7.4 Current Lightweight Regression Result

A fresh test run selected only tracked experiment-two tests and excluded unrelated untracked files. It executed 38 tests in 1.56 s: 36 passed and two failed. The failures are contract/source-text assertions rather than runtime simulation failures:

1. one test still expects the old sequence “initially close gripper before approach”, while the current required behavior opens at startup and closes only at the object;
2. one test expects vertical speed 8 deg/s, while the checked-in source currently uses 4 deg/s.

These failures identify stale tests that should be updated after the intended parameters are formally frozen. They also mean the repository cannot presently be reported as having a completely passing automated test suite.

Chapter 8 Physical Verification on RoboMaster EP

8.1 Implemented Physical Sequence

The physical program uses the RoboMaster EP’s planar arm and chassis rotation. Because this robot differs kinematically from the six-axis mechArm, the task is expressed as safe arm positions and chassis yaw:

1. park at (0, 150) mm and open the gripper;
2. rotate the chassis $\pm 45^\circ$ toward the source;

3. move the arm to (220, 0) mm and close;
4. park, rotate $\mp 90^\circ$ toward the destination, and descend;
5. open, park, rotate $\pm 45^\circ$ back to the initial direction;
6. reverse all rotation signs on the next cycle.

The simple five-cycle program executes exactly five attempts. The retry version instead counts five successful grasps and repeats a cycle after an inferred empty grasp. It also shows the 360p camera stream and supports E-key interruption.

8.2 Recorded Hardware Result

The file docs/videos/真机连续五次抓取(包含失败案例).mp4 has duration 186.87 s, 30.000 FPS, and 5,606 frames. It visibly records the RoboMaster EP, marked floor workspace, object, arm, gripper, and repeated chassis/arm motion. The recording intentionally includes a failure case. Representative frames in Figure 7 show the robot in safe/parked, approach, transport, and return configurations.

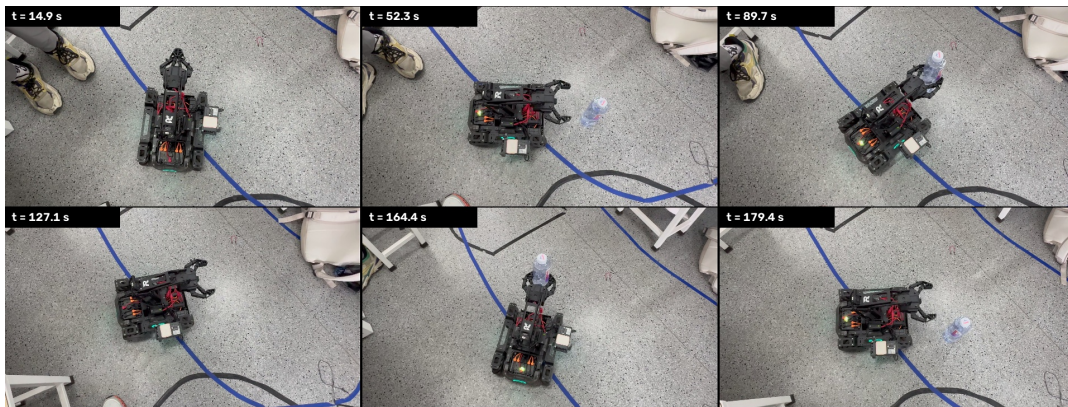


Figure 7 Representative frames from the physical RoboMaster EP five-cycle recording

The repository does not contain a timestamped table mapping each physical attempt to success/failure. Therefore the video verifies that the physical sequence was exercised and a failure was preserved, but it does not independently prove the guide’s exact “at least 4 of 5” count. The report does not infer an unsupported count from the filename alone.

8.3 Deviation from the Required Physical Platform

The guide specifies Jetson Orin and mechArm 270 for the physical stage. Those devices were not available for this course run, so the instructor approved DJI RoboMaster EP as the substitute. I completed the physical fixed-point task through the EP’s Windows Python SDK. The RoboMaster chassis performed the transfer rotation and its arm has different degrees of freedom and coordinates, so the ROS 2 interface is not identical to simulation. This is an approved equipment adaptation, not an unfinished physical demonstration.

Chapter 9 Reproducibility and Operating Procedure

9.1 Simulation Startup

From Windows PowerShell, the checked-in workflow is:

Listing 2 Build and start the simulation runtime

```

1 cd "E:\机器人集成小组项目\实验二"
2 .\scripts\start_moveit_sim.ps1
3
4 docker compose -f ".\docker-compose.yml" exec moveit bash -lc \
5 "source /opt/ros/humble/setup.bash && \
6 colcon --log-base /opt/mecharm_ws/log build \
7 --base-paths /workspace/mecharm_exp2/simulation/urdf/mycobot_description \
8 /workspace/mecharm_exp2/src \
9 --build-base /opt/mecharm_ws/build \
10 --install-base /opt/mecharm_ws/install --symlink-install"
```

After the Isaac timeline is playing and the TCP server is listening, the five-cycle task is:

Listing 3 Run five alternating simulation cycles

```

1 docker compose exec moveit bash -lc \
2 "source /opt/ros/humble/setup.bash && \
3 source /opt/mecharm_ws/install/setup.bash && \
4 ros2 launch mecharm_pick_place \
5 alternating_cartesian_pick_place.launch.py \
6 project_root:=/workspace/mecharm_exp2 cycles:=5"
```

The empty-grasp exception is launched by replacing the Launch filename with `empty_grasp_test.launch.py`. The expected terminal evidence is `EMPTY_GRASP_EXCEPTION code=NO_OBJECT` followed by `EMPTY_GRASP_RETURN_HOME`.

9.2 Physical-Robot Startup

The physical computer must first connect directly to the RoboMaster EP Wi-Fi. Then:

Listing 4 Prepare the verified RoboMaster SDK environment

```

1 conda activate robomaster
2 $env:RM_SDK_ROOT = "C:\Users\xhr\RoboMaster-SDK-fixed"
3 python -c "import os; root=os.environ['RM_SDK_ROOT']; \
4 keep=[os.add_dll_directory(os.path.join(root,'lib','libmedia_codec', \
5 'src','ffmpeg-dll')), os.add_dll_directory(os.path.join(root,'lib', \
6 'libmedia_codec','src','opus-dll'))]; exec(open( \
7 r'E:\机器人集成小组项目\实验二\robomaster_ep\scripts\ \
```

```
8 pick_rotate_place_5_cycles_retry.py',encoding='utf-8').read())"
```

Connection, small arm motion, gripper, chassis rotation, and camera tests should be run before the combined sequence. The motion script must not be run while another device controls the same robot.

9.3 Version-Control Evidence

The public repository is <https://github.com/xhr-CHN/mecharm-fixed-point-pick-place>. At report generation, main contains 31 commits. Representative milestones include initial documentation, ROS 2 task and Isaac integration, official adaptive-gripper assets, motion stabilization, MoveIt 2 configuration, TCP runtime, Cartesian IK, gripper loop repair, pre-grasp clearance tuning, RoboMaster scripts, retry/five-cycle programs, recordings, and bilingual README updates. Videos are managed through Git LFS because two files exceed GitHub's normal 100 MB file limit.

Table 7 Representative incremental commits

Commit	Type	Content
a25b4b2	feat	ROS 2 task and Isaac Sim integration
377ab33	feat	Official mechArm adaptive-gripper assets
4739d15	feat	Stable Isaac scene integration
5fb030b	feat	MoveIt 2 configuration
fd7be61	fix	End-to-end trajectory transport
0026efe	feat	Containerized TCP runtime
c5863e9	feat	Calibrated Cartesian grasping
8220502	fix	Adaptive-gripper loop physics and scene refresh
ac41e48	tune	Higher pre-grasp clearance
5140448	tune	Grasp pose and Isaac drive parameters
4e6f3d2	feat	Physical retry and multi-cycle scripts
50f8de2	feat	Alternating tests and experiment recordings
324ea5f	docs	Chinese and English operating instructions

Chapter 10 Discussion of Problems and Engineering Decisions

10.1 Problems Solved

The largest integration problems were not the high-level pick/place states but mismatches between model topology, middleware discovery, and dynamic control:

- The URDF omitted closed-loop gripper constraints, so the distal linkages were repaired with explicit PhysX joints.
- Multi-follower mimic behavior was unreliable, so the scalar command is mirrored at the transport boundary.
- Cross-system DDS discovery was unstable under TUN routing, so a dedicated TCP channel replaced it while ROS 2 remained inside the container.
- Cubic/direct target changes excited visible oscillation, so targets now use a quintic S-curve with separate slow vertical motion.
- The tool initially approached at an incorrect yaw/tilt, so the numerical solver now preserves verticality and applies yaw relative to the HOME tool axis only during grasping.
- The gripper descended into the block before closure, so closing height was raised above the object top and open/close settling waits were added.

10.2 Remaining Risks and Limitations

The main remaining simulation risks are inaccurate collision meshes, the fixed-joint grasp helper, and stale automated contracts. MoveIt/OMPL assets exist, but the recorded five-cycle success uses direct numerical IK rather than collision-aware planning. The fixed trajectory, execution outcomes, parameters, and errors are preserved in the delivery package. A duplicated `grasp_yaw_offset_deg` key also exists in the YAML file; both values are currently identical, but the duplicate should be removed to avoid ambiguous future edits.

The physical stage has the larger compliance gap. RoboMaster EP proves that the group operated a real mechanism and handled retries/safe exit, but it cannot validate the mechArm model, six-joint IK, ROS 2 controller, or simulation-to-hardware parameter-only switch. The close-time heuristic can confuse varying object width, friction, battery voltage, or stalled motion with successful grasping. A compliant revision should use physical mechArm feedback, a ROS 2 driver, explicit emergency-stop evidence, and machine-readable attempt logs.

Chapter 11 Conclusions and Recommended Completion Work

The experiment produced a functioning fixed-point simulation with a mechArm 270 Pi model, adaptive-gripper physics repair, fixed A/B points, safe approach/closing heights, vertical numerical IK, a 90° grasp-yaw offset, quintic 120 Hz joint interpolation, separate

normal/vertical speeds, and explicit error handling. A 385.57 s video records five successful alternating cycles, and a separate failure recording is retained. The ROS 2/Docker and Windows Isaac integration is reproducible through the checked-in startup scripts and TCP bridge.

Physical work on RoboMaster EP demonstrates repeated real grasp, chassis transfer, release, retry, camera observation, and safe cleanup. The instructor-approved platform substitution was necessary because the specified hardware was unavailable. The resulting task is complete on the available platform, while the different ROS 2/SDK interface remains an explicitly documented compatibility boundary.

The shortest path to complete strict acceptance is:

1. optionally add the recorder node to the alternating Cartesian Launch when a raw time-series trace is required; the fixed waypoint and result records are already preserved;
2. update the two stale contract tests and rerun the tracked test suite until all tests pass;
3. combine Isaac startup and ROS task launch behind one documented top-level entry, or obtain instructor approval for the current PowerShell-plus-Launch arrangement;
4. if the specified mechArm 270 and Jetson Orin become available later, validate the existing task logic through the corresponding device interface and calibrated position parameters;
5. record object mass, emergency-stop procedure, per-member safety checks, five attempt outcomes, and at least four successful placements.

With these items, the existing software foundation can be converted from a successful mixed-platform prototype into a fully evidenced course acceptance package.

A Submission File Cross-Reference

Table 8 Required materials and repository paths

Required material	Repository path/evidence
ROS 2 programs	src/mecharm_pick_place/, src/mecharm_moveit_demo/
Launch files	src/mecharm_pick_place/launch/ and src/mecharm_moveit_config/launch/
Parameter files	config/simulation.yaml; src/mecharm_pick_place/config/direct_cartesian_pick_place.yaml
Simulation model	simulation/urdf/mycobot_description/; simulation/scenes/mecharm_pick_place.usd
Pick/place/safe height	direct_cartesian_pick_place.yaml
Simulation videos	docs/videos/仿真连续五次抓取.mp4; failure-case video in same folder
Physical video	docs/videos/真机连续五次抓取（包含失败案例）.mp4
Physical programs	robomaster_ep/scripts/
Five-attempt result record	Fixed waypoint/result records and successful simulation video; physical video includes a failure and recovery
Exception record	Empty-grasp source/log contract and simulation failure video; standalone final error file still missing
Operating instructions	README.md, README_EN.md, docs/testing.md, robomaster_ep/README.md
Report	report/main.tex and compiled PDF

B Report Evidence Method

The report was assembled from the laboratory guide, current tracked source/configuration files, Git history, existing result directories, and the three preserved MP4 files. Contact sheets were generated by report/extract_video_frames.py; the script decodes fixed relative

timestamps and writes reproducible JPEG sheets. Video duration, FPS, and frame counts were read from the decoded streams. No result count was inferred where the repository lacked a per-attempt record.

References

- [1] Robot Systems Integration Laboratory. Mechanical Arm Fixed-Point Grasping (Group Experiment): Simulation Development and Physical Verification Requirements[S]. 2026.
- [2] Elephant Robotics. mycobot_ros2: myCobot ROS 2 Package[EB/OL]. Humble branch. https://github.com/elephantrobotics/mycobot_ros2.
- [3] NVIDIA Corporation. Isaac Sim Documentation, Version 5.1.0[EB/OL]. <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/>.
- [4] Open Robotics. ROS 2 Humble Documentation[EB/OL]. <https://docs.ros.org/en/humble/>.
- [5] PickNik Robotics. MoveIt 2 Documentation: Humble[EB/OL]. <https://moveit.picknik.ai/humble/>.
- [6] DJI. RoboMaster Python SDK Documentation[EB/OL]. <https://robomaster-dev.readthedocs.io/>.